

Graphon Conversational Memory Benchmark

LOCOMO and LongMemEval-S Evaluation Report

Prepared by Graphon AI | 2026-07-15 | LOCOMO n=300, LongMemEval-S n=50

Executive Summary

Graphon was evaluated on the two academic benchmarks the memory-systems field reports on: **LOCOMO** (n=300, multi-session conversational QA) and **LongMemEval-S** (n=50, ~115k-token chat histories per question). Answers were graded by key-fact coverage with auditable verbatim-quote verdicts.

On LOCOMO, Graphon answered directly at **85.2% QA accuracy** using Graphon's **ultra** reasoning mode (an agentic answer harness), with **retrieval recall@10 of 0.751** (measured on Graphon's standard retrieval).

On LongMemEval-S, Graphon answered directly at **79.5% QA accuracy** using Graphon's **ultra** reasoning mode (an agentic answer harness).

Graphon builds its memory index server-side: this harness spent **zero external LLM credits on ingestion**.

Results at a Glance

Suite	Dataset (n)	Metric	Graphon	Published field*
Memory	LOCOMO (300)	QA accuracy (ultra)	85.2%	graphify 45.3%, supermemory 49.7%, BM25 31.3%, mem0 27.3%
Memory	LOCOMO (300)	recall@10	0.751	graphify 0.497, BM25 0.362, supermemory 0.149, mem0 0.048
Memory	LongMemEval-S (50)	QA accuracy (ultra)	79.5%	graphify 76%, dense RAG 76%, BM25 70%, mem0 70%
Cost	graph build	external LLM credits	\$0	graphify \$0

* Published numbers are from the graphify benchmark page (2026-07-05), which ran mem0 and supermemory as adapters in its own harness with a Kimi K2.6 reader/judge. This report uses the same datasets, sample-size targets, pipeline shape, and key-fact coverage grading, with gpt-4o as the shared reader/judge; numbers are cross-referenceable, not identical-conditions. Our in-harness BM25 anchor (next section) calibrates the two harnesses.

LOCOMO (n=300)

Ten multi-session conversations (locomo10.json, snap-research/locomo); questions sampled with a fixed seed, stratified over the four scored categories (adversarial excluded, as the field reports). One Graphon group per conversation; the BM25 anchor indexes the identical rendered text and shares the same reader and judge. Graphon (direct) ran in **ultra** reasoning mode — an agentic answer harness inside Graphon rather than a fixed top-10 retrieval, so recall@10 for that row is measured on Graphon's standard retrieval (identical to the shared-reader row).

System	QA accuracy	recall@10	Avg latency
Graphon (direct answer, ultra)	85.2%	0.751	30.10s
Graphon + shared reader	64.7%	0.751	6.88s
BM25 anchor + shared reader	46.5%	0.553	0.68s

Anchor calibration. Judges and readers differ across harnesses, so absolute numbers shift together with the BM25 anchor. In this harness Graphon (direct) leads the shared BM25 anchor by **+38.7 points**; in the published graphify table, graphify leads its BM25 anchor by +14.0 points (45.3% vs 31.3%). Reading system-minus-anchor deltas keeps the comparison harness-independent.

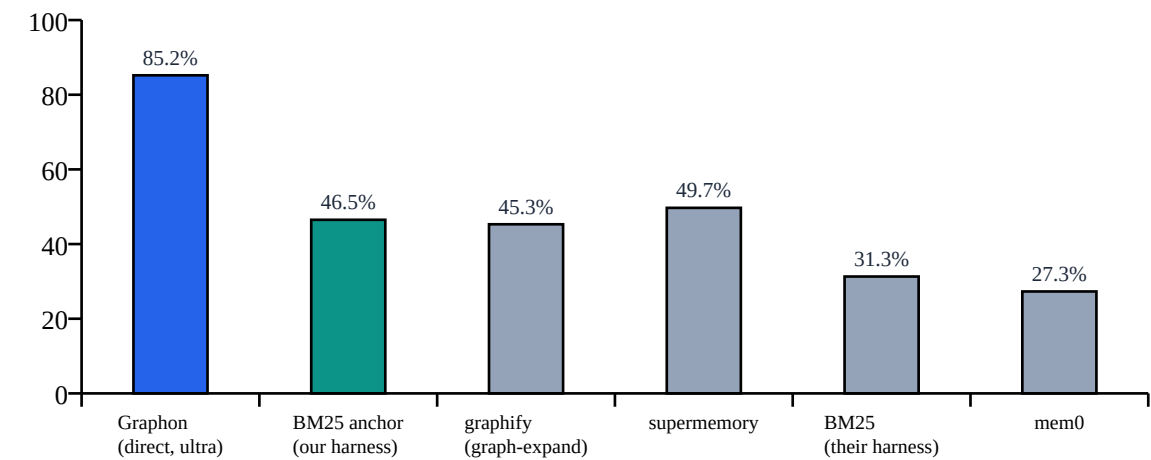


Figure 1. Locomo QA accuracy: Graphon and our BM25 anchor vs published vendor numbers (gray).

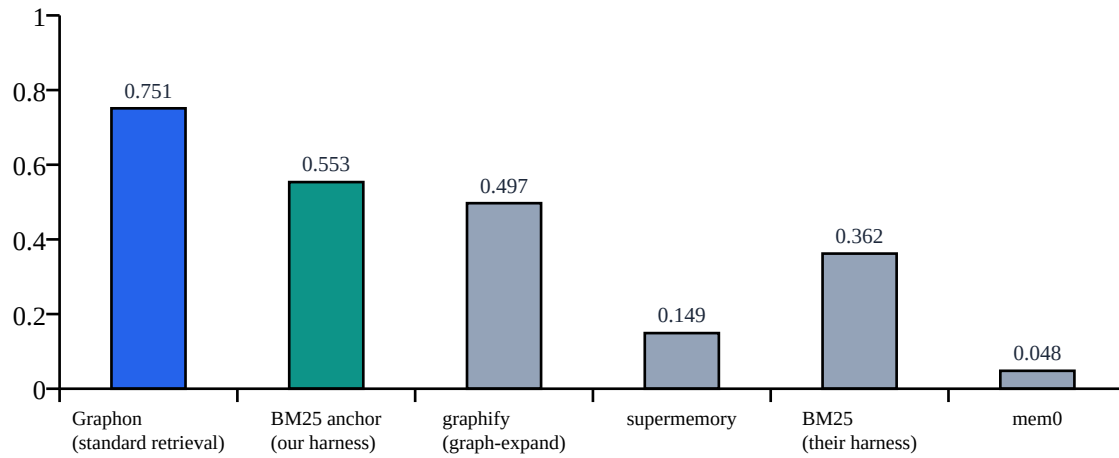


Figure 2. Locomo retrieval recall@10 (fraction of gold evidence turns present in the top-10 retrieved chunks).

LOCOMO accuracy by question category

Category	n	Graphon direct (ultra)	Graphon + reader	BM25 anchor
single-hop	164	92.1%	73.9%	60.3%
multi-hop	55	76.8%	57.3%	16.4%
temporal	62	80.2%	58.2%	50.0%
open-domain	19	66.5%	27.0%	2.6%

LongMemEval-S (n=50)

Each question carries its own ~115k-token multi-session chat history (~40-50 sessions). Every haystack session was ingested as its own timestamped document into a dedicated Graphon group (one group per question). Sample stratified by question type, abstention questions included. recall@10 is session-level: the fraction of gold evidence sessions among the sessions of the top-10 retrieved chunks. Graphon (direct) ran in **ultra** reasoning mode — an agentic answer harness inside Graphon rather than a fixed top-10 retrieval, so recall@10 is not reported for that row.

System	QA accuracy	recall@10	Avg latency
Graphon (direct answer, ultra)	79.5%	-	62.59s
Graphon + shared reader	47.0%	0.827	7.95s
BM25 anchor + shared reader	54.5%	0.923	0.75s

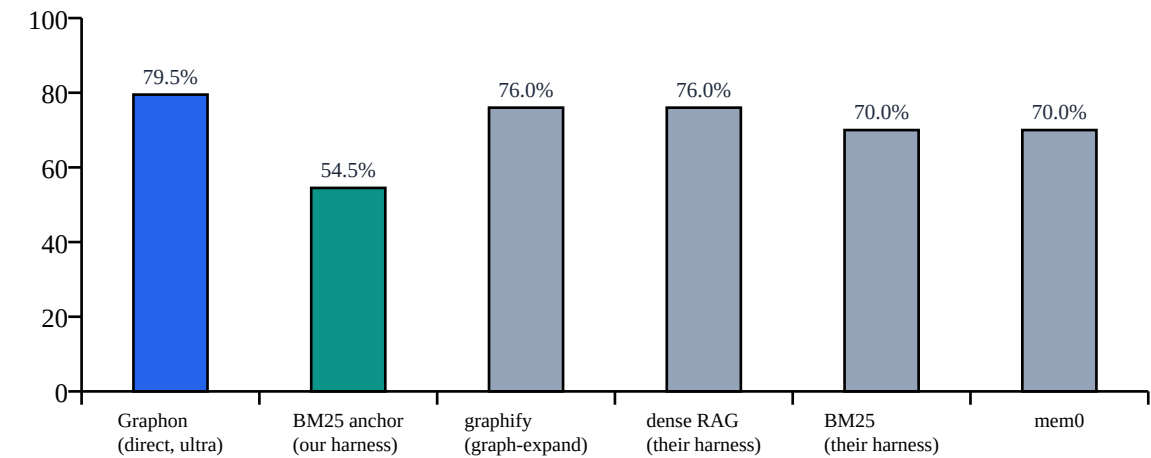


Figure 3. LongMemEval-S QA accuracy vs published numbers (gray).

Question type	n	Graphon direct	BM25 anchor
knowledge-update	8	75.0%	37.5%
multi-session	13	76.9%	50.0%
single-session-assistant	6	100.0%	83.3%
single-session-preference	3	58.3%	0.0%
single-session-user	7	100.0%	85.7%
temporal-reasoning	13	69.2%	51.9%

Methodology

1. Pipeline per question: ingest (render conversations to markdown) -> index (Graphon group / BM25) -> search (top-10) -> answer -> grade.
2. Graphon 'direct' uses Graphon's own end-to-end answer — how the product is used. 'Shared reader' has gpt-4o answer from Graphon's top-10 retrieved sources, structurally identical to how retriever-style harnesses run every system; both are reported.
3. One model (gpt-4o) fills every external LLM role: shared reader, key-fact decomposition, and judge. Temperature 0.
4. Grading is key-fact coverage: gold answers are decomposed into atomic facts (cached); the judge marks each fact covered / partial / missing and must cite a verbatim quote from the answer for every non-missing verdict. $\text{coverage} = (\text{covered} + 0.5 \times \text{partial}) / \text{total}$. QA accuracy is mean coverage.
5. No scoring metadata (dialog ids, gold session ids, has_answer flags) is ever present in the ingested text. Questions and gold answers never reach Graphon.
6. Runs are seeded, resumable, and write a per-role token/cost spend ledger; every row (answer, verdicts, quotes, latencies) is stored in JSONL for audit.

Cost

Item	Value
Graph/index build, external LLM credits	\$0.00 (Graphon indexes server-side)
LOCOMO run OpenAI spend (reader + judge)	\$8.24
LongMemEval-S run OpenAI spend (reader + judge)	~\$1.52
Reasoning effort	LOCOMO: ultra; LongMemEval-S: ultra
Shared reader / judge model	gpt-4o

Reproducing: see [experimental/memory_bench/README.md](#) — `download_data.py` fetches the pinned datasets; `run_eval.py --benchmark locomo|longmemeval` reruns the evaluation; `make_customer_report.py` rebuilds this report.